Aula 07 — Modelos de Fundação, Transformers e Engenharia de Prompts

Material de apoio · ISI022 — Sistemas de Informação e Tecnologias Emergentes

Apresentação

Este material acompanha a **Aula 07** e introduz modelos de linguagem usados em aplicações de Inteligência Artificial Generativa. A proposta é entender, sem exigir matemática avançada, como o texto é processado, como um Transformer ajuda a relacionar partes do contexto e como escrever instruções mais úteis para tarefas de Sistemas de Informação.

Modelos generativos podem resumir documentos, classificar solicitações, extrair campos e apoiar a redação. A fluência da resposta, porém, não comprova que ela seja verdadeira. Em processos organizacionais, o modelo deve ser avaliado com dados e critérios adequados, e as pessoas continuam responsáveis pelas decisões que afetam usuários e operações.

1. De inteligência artificial a modelos de fundação

Inteligência Artificial (IA) é um campo amplo que estuda sistemas capazes de executar tarefas associadas a capacidades como percepção, previsão, linguagem e tomada de decisão. **Aprendizado de máquina** é uma abordagem em que o sistema ajusta seus parâmetros a partir de exemplos. Redes neurais são uma família de modelos de aprendizado de máquina inspirada, de maneira abstrata, em unidades conectadas.

Um **modelo de fundação** é treinado em dados amplos e pode servir de base para muitas aplicações. Um **modelo de linguagem de grande porte (LLM)** trabalha com sequências de linguagem e pode gerar ou transformar texto. O termo “fundação” descreve seu uso amplo como ponto de partida; não significa que o modelo tenha acesso aos documentos privados de uma organização, conheça mudanças recentes ou esteja certo em cada resposta.

Uma aplicação de IA generativa normalmente reúne mais componentes que o modelo:

Componente	Função no sistema
Modelo	Calcula uma saída a partir da entrada e do contexto recebido.
Instruções	Definem a tarefa, os limites e o formato desejado.
Aplicação	Controla interface, permissões, validações e integração com outros sistemas.
Dados de contexto	Podem incluir texto fornecido pelo usuário ou fontes recuperadas pela aplicação.
Pessoa responsável	Revisa resultados e toma decisões quando o risco ou o processo exigir.

2. Como um modelo de linguagem produz texto

Uma visão simplificada da geração segue estas etapas:

1. Tokenização: o texto é dividido em unidades chamadas *tokens*. Um token pode corresponder a uma palavra, parte de uma palavra, pontuação ou outro fragmento. Por isso, token não é sinônimo de palavra.

2. Representação numérica: cada token é convertido em números que permitem ao modelo realizar cálculos. A posição dos tokens também é representada, pois a ordem altera o sentido.

3. Processamento contextual: camadas do modelo transformam as representações e calculam relações entre as partes da sequência.

4. Distribuição de probabilidades: para a geração de texto, o modelo calcula quais tokens poderiam vir a seguir e suas probabilidades condicionadas ao contexto.

5. Seleção e repetição: uma regra de geração escolhe o próximo token. Ele é acrescentado à sequência e o processo se repete até atingir um limite ou critério de parada.

Isso ajuda a explicar por que uma resposta pode parecer natural sem ter sido consultada em uma fonte oficial. O mecanismo estima continuações com base no que aprendeu e no contexto disponível. Ele não transforma automaticamente toda pergunta em uma busca factual.

Exemplo breve de previsão

Considere o início: “O sistema ficou indisponível porque…”. O modelo considera continuações possíveis segundo o contexto recebido. Se o único dado fornecido for “a página mostrou erro 503”, afirmar “por sobrecarga do banco de dados” acrescenta uma causa que não está demonstrada. Uma resposta prudente distingue o fato observado da hipótese e pede evidências adicionais.

3. Transformers e mecanismo de atenção

O artigo *Attention Is All You Need* apresentou a arquitetura Transformer, baseada em mecanismos de atenção e amplamente adotada em modelos de linguagem. Em termos intuitivos, a **atenção** permite que o modelo pondere as relações entre diferentes posições da sequência ao construir representações contextuais.

Na frase “A gerente avisou a analista que ela deveria revisar o relatório”, interpretar “ela” exige relacionar palavras distantes e o contexto. A atenção oferece um mecanismo para calcular relações desse tipo. Ela não é, por si só, uma explicação legível do raciocínio, nem uma prova de que a interpretação está correta.

Em modelos que geram texto da esquerda para a direita, uma **máscara causal** evita que a previsão de um token use tokens futuros. A cada etapa, o modelo usa o prefixo já disponível para estimar a continuação. A arquitetura original também descreve codificadores e decodificadores; diferentes famílias de modelos usam partes e variantes distintas desse desenho.

4. Treinamento, inferência e parâmetros de geração

- **Pré-treinamento:** o modelo ajusta seus parâmetros a partir de grandes coleções de dados, aprendendo padrões úteis para tarefas de linguagem.
- **Ajuste ou alinhamento:** etapas adicionais podem adaptar o comportamento para seguir instruções, preferências ou critérios específicos.
- **Inferência:** o sistema recebe uma entrada e calcula uma resposta usando o modelo já treinado.

Essas etapas não devem ser confundidas. Durante a inferência, o modelo não necessariamente atualiza seus parâmetros ou passa a conhecer automaticamente documentos que não foram fornecidos à aplicação.

O contexto disponível tem limite, assim como tempo e recursos computacionais. Configurações de geração podem afetar diversidade e repetibilidade, mas não são um botão de veracidade. Temperatura, por exemplo, influencia a distribuição usada para selecionar tokens em implementações que a oferecem; ajustar esse valor não substitui fontes, testes ou revisão.

5. Engenharia de prompts como especificação

Um **prompt** é a entrada que orienta a execução da tarefa. Para tornar uma solicitação mais clara, especifique:

1. Tarefa e público: qual resultado é necessário e para quem?

2. Contexto e dados permitidos: que informações a resposta pode usar?

3. Formato: lista, tabela, campos estruturados, idioma ou tamanho.

4. Critérios: como reconhecer uma saída útil e completa?

5. Limites e incerteza: o que fazer se faltar informação? O que não pode ser inventado ou solicitado?

Exemplo: prompt vago e prompt especificado

Vago: “Analise este chamado e diga o que fazer.”

Mais específico:

> Classifique o chamado fictício abaixo para a equipe de suporte. Use somente os fatos informados. Retorne os campos `categoria_provisoria`, `evidencias`, `informacoes_ausentes` e `proximo_passo_seguro`. Não determine uma causa técnica nem prometa prazo sem evidência. Se não houver dados suficientes, indique “insuficiente” e formule uma pergunta de esclarecimento. Não solicite senhas.

O segundo prompt define a tarefa, a fonte permitida, a estrutura da saída e o comportamento esperado diante de lacunas. Isso melhora a possibilidade de avaliar a resposta, mas ainda não garante que todos os campos estejam corretos.

Exemplo de saída estruturada

Para integrar uma aplicação, pode ser útil pedir uma estrutura com campos conhecidos. Exemplo ilustrativo:

```
{
  "categoria_provisoria": "indisponibilidade de serviço",
  "evidencias": ["relato de erro 503", "mais de uma pessoa afetada"],
  "informacoes_ausentes": ["horário exato", "serviços afetados"],
  "proximo_passo_seguro": "confirmar o alcance e encaminhar para a equipe responsável"
}
```

Uma aplicação real deve validar a estrutura e os valores antes de usá-los. JSON sintaticamente válido não significa que o conteúdo esteja correto, autorizado ou adequado ao processo.

6. Exemplo aplicado a Sistemas de Informação

Uma equipe de suporte recebe o seguinte chamado fictício:

> “Desde hoje cedo, não consigo acessar o sistema de pedidos. A página fica carregando e depois aparece erro 503. Outros colegas também relataram. Tenho uma apresentação às 14h.”

Uma triagem apoiada por IA poderia organizar os dados, mas deve preservar a diferença entre:

Tipo	Exemplo no caso	Como tratar
Fato informado	A página exibiu erro 503; outras pessoas também relataram dificuldade.	Registrar como relato, sem assumir que todos viram o mesmo erro.
Hipótese	O serviço pode estar indisponível para mais usuários.	Marcar como hipótese a investigar.
Dado ausente	Causa, impacto total, horário preciso e duração.	Solicitar informação ou consultar ferramenta autorizada.
Decisão operacional	Prioridade final e comunicação de prazo.	Seguir política institucional e manter responsável humano.

O modelo pode ajudar a estruturar uma resposta inicial. A aplicação deve evitar expor dados pessoais, limitar o acesso, validar a saída e deixar claro quem responde pela decisão. Se a tarefa depender de procedimentos internos atualizados, será necessário fornecer ou consultar uma fonte institucional autorizada.

7. Limites e boas práticas

- **Erro factual:** verifique afirmações importantes em documentos ou sistemas de referência.
- **Viés:** examine se respostas variam injustamente entre grupos ou situações semelhantes.
- **Privacidade e confidencialidade:** não envie dados pessoais, segredos, credenciais ou informações internas a serviços sem autorização e controles adequados.
- **Injeção de instruções:** trate texto recebido de usuários ou documentos como dado não confiável; ele não deve substituir as regras da aplicação.
- **Validação:** confira estrutura, conteúdo, permissões e regras de negócio antes de integrar uma saída a ERP, CRM ou outro SI.
- **Responsabilidade:** defina quando uma pessoa deve revisar, aprovar ou rejeitar a recomendação.

Use a IA como apoio compatível com o risco da tarefa. Resumos de baixo impacto ainda podem conter erros; decisões sobre acesso, segurança, pessoas, pagamentos ou continuidade operacional exigem controles proporcionais.

Referências para estudo

- VASWANI, A. et al. Attention Is All You Need. NeurIPS, 2017.
- OPENAI. Prompt engineering. Documentação oficial.
- STAIR, R. M.; REYNOLDS, G. W. *Princípios de Sistemas de Informação*. Thomson Pioneira, 2013.

Exercícios

Resolva individualmente ou em dupla. Justifique as respostas com base no material.

1. Explique a diferença entre modelo de fundação, LLM e aplicação de IA generativa.

2. Ordene e descreva as etapas: representação numérica, seleção do próximo token, tokenização, processamento contextual e repetição da geração.

3. O que a atenção permite ao Transformer? Por que ela não deve ser tratada como evidência de compreensão ou de correção?

4. Diferencie pré-treinamento e inferência. O que uma aplicação precisa fazer para usar documentos internos atualizados?

5. Um modelo atribui o erro 503 a uma falha no banco de dados. Que parte do caso sustenta essa conclusão? Que dados seriam necessários para confirmá-la?

6. Reescreva o prompt “Leia este e-mail e abra um chamado” especificando tarefa, informações permitidas, formato de saída e comportamento quando faltarem dados. Não use informações pessoais reais.

7. Para o exemplo de chamado da seção 6, crie uma tabela com três afirmações possíveis e classifique cada uma como fato informado, hipótese ou sem suporte.

8. Indique três riscos de usar uma saída generativa em um processo organizacional e proponha um controle para cada risco.

9. Que verificações uma aplicação deve fazer antes de gravar uma resposta gerada em um ERP ou CRM?

10. Elabore uma regra curta que determine quando a triagem pode ser assistida por IA e quando uma pessoa deve aprovar a decisão.